

10 PostgreSQL Internals

Understanding PostgreSQL internals is what separates senior engineers from staff and principal engineers. Anyone can write queries and tune indexes with a reference guide, but staff-level engineers reason from first principles: they know why a query plan changed after a `VACUUM`, why `FILLFACTOR` prevents index bloat, and what `t_xmin` / `t_xmax` actually mean on disk. This folder covers the storage engine, MVCC visibility, WAL durability, `VACUUM` mechanics, query planning internals, and system catalog structure — the substrate that everything else in PostgreSQL is built on.

Contents

Filename	Topic	Difficulty
12_internals_interview_guide.md	50 deep-dive Q&A: heap pages, MVCC, TOAST, WAL, VACUUM, checkpoints, query pipeline, system catalogs	Hard / Staff

Learning Path

Suggested reading order with rationale:

- Heap page layout (Q1–Q8)** — Start here. Every other concept (MVCC, VACUUM, TOAST) is meaningless without understanding the 8 kB page structure, `ItemId` array, and tuple header fields. Read these first even if you feel rushed.
 - MVCC visibility (Q9–Q16)** — Build directly on page layout. The `xmin/xmax` fields you learned in step 1 are the foundation of the snapshot-based visibility rules here. Cover the HOT update chain (Q11) and `XID` wraparound (Q13) before moving on.
 - TOAST (Q17–Q22)** — Relatively self-contained. Understand why the 2 kB threshold exists, how storage strategies differ, and what the 18-byte toast pointer encodes. This is frequently tested in staff interviews because few candidates know the chunk structure.
 - WAL (Q23–Q32)** — Central to durability, replication, and PITR. Study the WAL record structure (Q24) before the crash recovery mechanism (Q30). Understanding full-page writes (Q25) unlocks why `wal_compression` matters.
 - VACUUM (Q33–Q40)** — Builds on MVCC (you need to understand visibility horizons to understand why VACUUM cannot remove certain dead tuples). Cover the four VACUUM phases (Q33), the visibility map (Q34), and `XID` wraparound prevention (Q36) as a connected unit.
 - Checkpoints (Q41–Q44)** — Shorter section. Understand the purpose (bound crash recovery time), the two phases (write + `fsync`), and the interaction with `max_wal_size` to prevent forced checkpoints.
 - Query pipeline (Q45–Q50)** — End here. The pipeline builds on everything: statistics quality depends on `ANALYZE` (part of VACUUM), plan quality depends on understanding `random_page_cost` for storage hardware, and parallel query internals build on process-model knowledge.
-

Prerequisites

Before working through this folder, you should already be comfortable with:

- **Transactions and ACID** — Know what atomicity, isolation, and durability mean in practice; understand commit, rollback, and savepoints.
- **MVCC basics** — Have a working understanding that PostgreSQL uses multiversion concurrency (multiple row versions) rather than locking for reads. You do not need to know the implementation details — those are covered here.
- **Basic query optimization** — Be able to read EXPLAIN output, understand sequential scan vs. index scan cost trade-offs, and know what ANALYZE does at a high level.
- **Index types** — Know that B-tree, Hash, GIN, and GiST indexes exist and have different use cases. Index internals are not covered in this folder but are referenced.

Key Concepts Quick Reference

Concept	One-Line Explanation
PageHeaderData	24-byte struct at the start of every 8 kB heap page storing LSN, pd_lower, pd_upper, and flags
ItemId	4-byte line pointer in the page's item array encoding tuple offset, length, and status flags
HeapTupleHeaderData	24-byte tuple header storing t_xmin, t_xmax, t_ctid, infomask bits, and null bitmap
t_xmin / t_xmax	Transaction IDs of the inserting and deleting/locking transaction for an individual tuple version
t_ctid	Current TID pointing to the latest version of a tuple; used to traverse HOT update chains
MVCC Snapshot	Point-in-time view of committed transactions captured as (xmin, xmax, xip) triple
HOT Update	Heap-only tuple update that avoids index modification when indexed columns are unchanged
TOAST	The Oversized-Attribute Storage Technique; stores values > ~2 kB as chunks in a companion table
LSN	Log Sequence Number; 64-bit byte offset into the WAL stream used for crash recovery and replication
Full-Page Write	Complete 8 kB page copy embedded in WAL on first modification after a checkpoint; prevents torn-page corruption
FrozenTransactionId	Special XID value (2) assigned to old tuples by VACUUM FREEZE to prevent XID wraparound
Visibility Map	Per-page bitfield tracking all-visible and all-frozen status; enables VACUUM skipping and index-only scans
Free Space Map	Binary max-heap tree recording free bytes per page; enables O(log N) page selection for inserts

Checkpoint	Background process that periodically flushes all dirty shared buffers and fsyncs data files
pg_xact	SLRU directory storing 2-bit commit status (in-progress/committed/aborted) for every transaction ID

Related Folders

- **09_Transactions_Concurrency** — Covers ACID semantics, isolation levels, deadlock detection, and advisory locks from the application perspective. Complements the storage-level MVCC coverage here.
- **08_Query_Optimization** — Covers EXPLAIN/ANALYZE interpretation, index strategy, and query rewriting. The cost model parameters documented in Q47 of this folder directly explain plan choices covered there.
- **12_Production_PostgreSQL** — Covers operational topics: connection pooling, monitoring, alerting, and maintenance runbooks. The autovacuum tuning knowledge from Q35–Q40 of this folder is prerequisite for the production maintenance material there.